Qwen

Архитектура Рэто **v10:** Стратегия превосходства над **Adam** для **CPU-**обучения на базе **AMD Zen**

Архитектурный Анализ и Синтез Компонентов **Adam** для ЦП

Для создания оптимизатора Рэто v10, способного превзойти Adam в условиях обучения на центральном процессоре (ЦП) с ограниченной оперативной памятью (ОЗУ), необходимо глубокое понимание как самой архитектуры Adam, так и ее ограничений в данном контексте. Adam, или "адаптивное оценивание моментов", является одним из самых популярных и успешных оптимизаторов в машинном обучении благодаря своей способности адаптировать размер шага для каждого параметра независимо [1][2][2][3]. Это достигается за счет комбинации двух мощных идеологий: момента и RMSProp [2][5]. Однако, несмотря на свою популярность, Adam имеет свои недостатки, особенно при работе с ограниченными ресурсами, что открывает возможности для улучшений в рамках Рэто v10.

Основная сила Adam заключается в его ядре, состоящем из нескольких ключевых компонентов. Во-первых, это экспоненциально взвешенная скользящая средняя градиентов, известная как первый момент (m_t). Этот компонент действует аналогично классическому моменту в оптимизации, добавляя "импульс" к обновлению весов, что помогает преодолевать локальные минимумы и сглаживать колебания градиентов, особенно в шумных данных [2][1]. Во-вторых, Adam использует вторую момент (v_t), которая представляет собой скользящее среднее квадратов градиентов. Это позволяет нормализовать размер шага для каждого параметра, делая его менее чувствительным к масштабу градиентов различных функций [1][6]. Такой подход позволяет Adam эффективно работать на сложных, искаженных поверхностях потерь, где градиенты имеют очень разный масштаб [8].

Однако простого применения этих двух моментов недостаточно. Adam также включает два важнейших механизма, которые обеспечивают его стабильность и производительность. Первый — это коррекция смещения. Поскольку моменты m_t и v_t инициализируются нулями, на ранних этапах их значения будут занижены. Коррекция смещения исправляет эту проблему, масштабируя моменты обратно к их несмещенным оценкам, что гарантирует, что начальные шаги не будут чрезмерно малыми [242][244]. Второй механизм — это декуплированное затухание весов (Weight Decay). В отличие от классического L2-регуляризации, которая добавляется к градиенту, AdamW применяет затухание напрямую к весам модели. Это предотвращает взаимодействие между весовым затуханием и адаптивными скоростями обучения, что приводит к более стабильному обучению и лучшим результатам в обобщении, особенно для больших моделей [111][324][377].

Несмотря на эти преимущества, применение Adam в условиях ограниченной ОЗУ на CPU сталкивается с рядом проблем. Во-первых, Adam требует хранения дополнительных состояний: первого и второго моментов (m_t и v_t), которые имеют тот же размер, что и сами веса модели. Для модели с миллиардами параметров, даже в формате одинарной точности (FP32), это означает дополнительное использование памяти в размере ~4 байта на параметр, что может удваивать объем памяти, необходимой для хранения состояния оптимизатора [194]. Хотя существуют методы сокращения, такие как 8-битный Adam, они могут снижать точность [18][195]. Во-вторых, стандартная реализация Adam может быть неэффективной с точки зрения использования кэша на CPU. Алгоритм требует множественных проходов по данным, что может приводить к плохой локальности и частым промахам в кэше, замедляя вычисления [147]. Наконец, Adam может иметь проблемы со сходимостью в некоторых выпуклых случаях, хотя эта проблема была частично решена в вариантах, таких как AMSGrad [114][115].

Рэто v10 должен взять за основу все проверенные компоненты Adam, но реализовать их в более эффективной и адаптированной форме. Ядро Рэто v10 будет состоять из тех же первых и вторых моментов, но с несколькими ключевыми отличиями. Во-первых, для решения проблемы памяти будет активно использоваться INT8 квантование обновлений. Это позволит сократить объем хранимых данных в четыре раза по сравнению с FP32, что является критически важным для помещаемых в ОЗУ моделей [233]. Для компенсации потери точности при квантовании будет реализован механизм обратной связи ошибок, который сохраняет и добавляет к следующему шагу округленную ошибку, обеспечивая

тем самым стабильность обучения [199]. Во-вторых, для оптимизации использования кэша будет применен тайлинг — разделение больших матриц на меньшие блоки, которые полностью помещаются в быструю кэш-память (L1/L2), что значительно ускоряет матричные операции на CPU [36]. В-третьих, для управления памятью будут использоваться специализированные аллокаторы, такие как bump allocator, который позволяет выполнять быстрое выделение памяти в рамках одного контекста без накладных расходов на освобождение, что идеально подходит для временных буферов, таких как активации и градиенты [129][319].

Более того, Рэто v10 должен унаследовать и усовершенствовать другие компоненты Adam. Коррекция смещения останется обязательной, поскольку она критически важна для стабильности на ранних этапах обучения [242]. Декуплированное затухание весов также будет интегрировано, поскольку оно доказало свою эффективность в улучшении обобщающей способности моделей [378]. Однако, чтобы достичь превосходства, Рэто v10 должен пойти дальше простого копирования. Он будет интегрировать новые математические идеи, такие как естественные градиенты и энтропийная регуляризация, которые модифицируют саму динамику обновления весов, делая ее более информативной и стабильной. Таким образом, архитектура Рэто v10 будет представлять собой гибрид: ядро Adam, оптимизированное для CPU, дополненное передовыми математическими инновациями и механизмами управления ресурсами для работы в условиях жестких ограничений по памяти.

Компонент	Реализация в **Adam**	Улучшение в Рэто **v10**
Первый момент **(Momentum)**	Скользящее среднее градиентов [21]	Использование для импульса; возможно, Nesterov lookahead [107]
Второй момент **(Adaptive LR)**	Скользящее среднее квадратов градиентов [116]	Использование для адаптации шага; возможно, K-FAC-like аппроксимация [29]
Bias Correction	Обязательная коррекция смещения моментов [242]	Сохранена как обязательный компонент для стабильности
Decoupled Weight Decay	Применение L2-регуляризации отдельно от градиента [324]	Сохранено для улучшения обобщения [377]
Хранение состояний	Два массива моментов такого же размера, как и веса [194]	Использование INT8 квантования с обратной связью об ошибке [199]
Использование кэша	Не оптимизировано специально для CPU	Применение тайлинга для улучшения локальности данных [36]
Управление памятью	Стандартные аллокаторы	Использование bump-аллокаторов для временных буферов [129]

Этот синтетический подход, сочетающий силу проверенного ядра Adam с инновационными технологиями для CPU и управления памятью, заложит прочный фундамент для достижения цели исследования — создания оптимизатора, который не просто конкурирует с Adam, но и превосходит его в специфических условиях обучения на центральном процессоре.

Интеграция Дифференциальной Геометрии и Теории Информации

Для достижения превосходства над Adam, Рэто v10 должен выходить за рамки простого адаптивного масштабирования градиентов и интегрировать более глубокие математические принципы, лежащие в основе дифференциальной геометрии и теории информации. Эти области предлагают мощные инструменты для понимания и ускорения процесса оптимизации, позволяя сделать шаги, которые являются не только адаптивными, но и геометрически и информационно осмысленными.

Дифференциальная геометрия предоставляет язык для анализа неевклидовых пространств, что особенно актуально для пространства параметров нейронных сетей. Естественный градиент — одно из ключевых понятий в этой области [27]. В отличие от обычного градиента, который указывает направление наибольшего увеличения функции в евклидовом пространстве, естественный градиент учитывает внутреннюю геометрию пространства параметров, заданную метрикой. Наиболее распространенной метрикой является информация Фишера (Fisher Information Matrix, FIM), которая определяет расстояние между двумя вероятностными распределениями, параметризуемыми весами модели [28]. Шаг в направлении естественного градиента является инвариантным к параметризации модели, что означает, что он зависит только от изменении в пространстве вероятностных распределений, а не от способа, которым эти распределения параметризуются. Это приводит к более эффективным путям сходимости и может значительно ускорить обучение, особенно на плохо условленных задачах [9].

Прямая вычислительная стоимость информации Фишера является главным препятствием для ее широкого применения. FIM является матрицей размера $N \times N$, где N — количество параметров модели, что делает ее вычисление и

хранение непрактичным даже для относительно небольших моделей [5]. Поэтому ключевой задачей для Рэто v10 является разработка эффективной аппроксимации FIM. Одним из наиболее успешных подходов является Kronecker-Factored Approximate Curvature (K-FAC) [29]. K-FAC аппроксимирует FIM для каждого слоя нейронной сети как произведение Кронекера двух меньших матриц, связанных с входами и выходами этого слоя. Это позволяет снизить сложность от $O(N^2)$ до $O(n_{in}^2+n_{out}^2)$, где n_{in} и n_{out} — размеры входа и выхода, что делает вычисления осуществимыми [31] [32]. Вариации K-FAC, такие как GaLore, предлагают еще более экономичные способы работы с низкоранговыми приближениями градиентов, что также может быть интегрировано в Рэто v10 [391][392]. Вместо полного вычисления K-FAC, Рэто v10 может использовать низкоранговую проекцию градиента, как в GaLore, что эквивалентно движению в пространстве, определенном приближением FIM [95]. Этот подход, известный как Gradient Low-Rank Projection, позволяет уменьшить требования к памяти, сохраняя при этом большую часть преимуществ естественного градиента [391].

Теория информации предлагает второй мощный набор инструментов для улучшения оптимизации — энтропийную регуляризацию. Идея заключается в добавлении в функцию потерь дополнительного члена, который штрафует модель за слишком "острую" или детерминированную выходную вероятность. Формально, это выглядит как добавление термина $-\alpha H(p_\theta(x))$, где H — энтропия, $p_\theta(x)$ — распределение вероятностей, предсказанное моделью, а α — коэффициент регуляризации [100]. Максимизация энтропии побуждает модель быть "неуверенной" в своих предсказаниях, что помогает избежать переобучения и улучшает обобщающую способность, особенно в условиях дефицита данных [181]. Этот подход тесно связан с принципом максимальной энтропии и используется в таких областях, как обучение с подкреплением и вариационные автокодировщики [366].

В контексте оптимизатора, энтропийная регуляризация влияет на градиент, который теперь должен учитывать не только градиент исходной функции потерь, но и градиент энтропийного штрафа. Для дискретного случая с распределением категорий это означает, что градиент будет смещен в сторону увеличения энтропии. Это можно интерпретировать как добавление "естественного" вектора градиента, направленного в сторону более равномерного распределения вероятностей. Интересно, что энтропийная регуляризация может улучшать сходимость и других оптимизационных

алгоритмов, таких как Natural Policy Gradient (NPG), и может рассматриваться как способ ускорения сходимости [183]. Кроме того, исследуется связь между энтропией градиента и обобщающей способностью модели. Увеличение энтропии градиентов может способствовать более разнообразным обновлениям весов, что помогает модели избегать переобучения [182].

Рэто v10 будет уникальным именно благодаря синтезу этих трех идей: ядра Adam, низкоранговой аппроксимации естественного градиента (по аналогии с K-FAC/GaLore) и энтропийной регуляризации. Формула обновления для Рэто v10 будет выглядеть следующим образом:

$$w_{t+1} = w_t - \eta_t \cdot \frac{\hat{m}<emw>t}{\sqrt{\hat{v}_t} + \epsilon} \circ (\nabla(I + \beta \cdot G_t)$$

} \mathcal{L} + \lambda \cdot \nabla_{w} S_{ent}) \cdot P_{low-rank

Где:

- w_t — веса на шаге t.
- η_t — скорректированная скорость обучения.
- $\hat{m}_t, \hat{v}_t$ — скорректированные (bias-corrected) первые и вторые моменты.
- $\mathcal{L}$ — основная функция потерь.
- $\nabla_w S_{ent}$ — градиент энтропийного регуляризатора.
- λ — коэффициент энтропийной регуляризации.
- $P_{low-rank}$ — оператор низкоранговой проекции градиента.
- G_t — матрица, аппроксимирующая информацию Фишера.
- β — гиперпараметр, контролирующий влияние геометрической поправки.
- $\circ$ — поэлементное умножение.

Такая формула сочетает в себе адаптивность Adam, стабилизирующую и улучшающую обобщающую способность энтропийную регуляризацию, а также геометрически инвариантный шаг, обеспечиваемый низкоранговой аппроксимацией естественного градиента. Это создает многоуровневую систему, которая не только адаптирует размер шага, но и направляет его в более осмысленном геометрическом и информационном пространстве. Реализация такой системы в Рэто v10 позволит ему не просто повторять успехи Adam, но и находить пути сходимости, которые недоступны для стандартных адаптивных оптимизаторов, особенно в условиях ограниченных ресурсов, где каждая итерация должна быть максимально эффективной.

Аппаратная Оптимизация для **AMD Zen**-Архитектуры

Для достижения превосходства над Adam в условиях ограниченной ОЗУ на центральном процессоре, Рэто v10 должен быть не просто математически совершенным, но и глубоко интегрированным с аппаратной архитектурой целевого оборудования — AMD Zen. Современные процессоры, такие как Ryzen, представляют собой сложные системы с многоуровневой иерархией кэшей, сложной топологией памяти (NUMA) и высокопроизводительными SIMD-инструкциями. Эффективное использование этих ресурсов является ключом к получению максимальной производительности. Рэто v10 будет построен вокруг трех столпов аппаратной оптимизации для AMD Zen: управление NUMA, оптимизация кэш-иерархии и использование SIMD-инструкций.

Первым и наиболее критичным аспектом является работа с архитектурой NUMA (Non-Uniform Memory Access). На многихсокетных системах доступ к памяти, расположенной на другом сокете, значительно медленнее, чем к локальной памяти [212]. Даже на односокетных процессорах, таких как Ryzen, существует NUMA-подобная топология, где ядра разделены на группы (CCX), и межгрупповой доступ к памяти также имеет повышенную задержку [202]. Незнание и игнорирование этой топологии приводит к значительному падению производительности, так как операции чтения/записи данных могут блокировать выполнение потоков на длительное время, пока данные загружаются из удаленной памяти [251]. Для Рэто v10 это означает необходимость принудительного управления политиками выделения памяти. В Linux это достигается с помощью утилиты `numactl`. Библиотека `numactl-rs` для Rust предоставляет удобный интерфейс для интеграции этих возможностей в код [204]. Рэто v10 будет использовать `numactl` для привязки процесса обучения к определенному NUMA-узлу (например, к одному CCX на Ryzen 8-ядерном процессоре). Это гарантирует, что все выделенные для модели и оптимизатора буферы (веса, градиенты, моменты) будут располагаться в локальной памяти, что минимизирует задержки и обеспечивает предсказуемую производительность [210][254]. Это особенно важно в многопоточном режиме, когда несколько потоков одновременно обращаются к общим данным.

Вторым столпом является оптимизация использования кэш-иерархии. Современные процессоры имеют несколько уровней кэша (L1, L2, L3), причем каждый последующий уровень больше, но медленнее. Количество промахов в кэше является главным фактором, ограничивающим производительность на CPU [147]. Неправильное управление памятью, такое как неэффективное хранение

матриц или непоследовательный доступ к элементам, приводит к тому, что данные постоянно "перескакивают" между уровнями кэша и основной памятью, вызывая огромные задержки. Для Рэто v10 будут применены две основные техники. Первая — это использование плоских массивов. Вместо вложенных структур данных, таких как `Vec<Vec<f32>>`, для представления матриц будут использоваться одномерные векторы `Vec<f32>`. Это гарантирует, что все элементы матрицы хранятся в одном непрерывном блоке памяти, что значительно улучшает локальность данных и повышает вероятность попадания в кэш при последовательном доступе [40] [42]. Вторая техника — это тайлинг. При выполнении больших матричных операций, таких как умножение или свертка, матрицы разбиваются на меньшие блоки (tiles), размер которых точно соответствует размеру кэша L1/L2. Затем операция выполняется поэтапно, обрабатывая один tile за другим. Это позволяет держать все необходимые данные для текущего расчета в быстрой кэш-памяти, минимизируя обращения к медленной основной памяти [36]. Реализация этих техник потребует изменения базовых операций над тензорами в фреймворке Ryza, но выигрыш в производительности будет ощутимым.

Третий столп — это использование SIMD (Single Instruction, Multiple Data) инструкций. Современные процессоры Zen поддерживают AVX-512, которые позволяют выполнять одну и ту же операцию (например, сложение или умножение) над несколькими числами с плавающей запятой одновременно [262] [326]. Это дает потенциальный ускоритель в 2, 4 или даже 8 раз в зависимости от инструкции и типа данных. Для Рэто v10 это означает, что самые часто используемые операции, такие как вычисление градиентов, обновление моментов и нормализация, должны быть явно оптимизированы с использованием SIMD. В Rust для этого можно использовать библиотеку `packed_simd`, которая предоставляет типы данных и операции для работы с SIMD-регистрами в безопасном, но высокоэффективном коде [54]. Например, вместо цикла, последовательно умножающего два массива, можно написать функцию, которая загружает данные в SIMD-регистры и выполняет параллельное умножение. Это требует более продуманного дизайна алгоритмов, но для задач, подобных умножению матриц, это абсолютно необходимо для достижения максимальной производительности [178]. AMD Zen 5, например, имеет 512-битный datapath для AVX-512, что еще больше усиливает этот аспект [230].

Наконец, необходимо учитывать особенности самой архитектуры Zen. Zen 5, например, демонстрирует значительные улучшения в пропускной способности

памяти и IPC (инструкций за такт) [286][332]. Также стоит учитывать улучшения в управлении тепловыделением и повышение эффективности [229]. Рэто v10 будет разрабатываться с учетом этих характеристик, чтобы максимально использовать возможности нового поколения процессоров. Например, более высокая пропускная способность памяти позволяет эффективнее использовать более сложные, но более точные методы аппроксимации, такие как K-FAC, которые ранее могли быть слишком медленными из-за ограничений по памяти. Таким образом, аппаратная оптимизация для AMD Zen — это не просто набор трюков, а фундаментальный подход к проектированию Рэто v10, который обеспечивает его способность работать быстро и эффективно именно на целевом оборудовании, предоставляя конкурентное преимущество перед универсальными оптимизаторами, не учитывающими особенности архитектуры.

Технология	Проблема	Решение в Рэто **v10**	Библиотеки
NUMA-Awareness	Высокая задержка доступа к удаленной памяти.	Принудительная привязка процесса к NUMA-узлу для локального выделения памяти.	`numactl-rs` [204], `numa-rs` [203]
Cache Optimization	Частые промахи в кэше из-за плохой локальности данных.	Использование плоских массивов и кэш-тайлинг для матричных операций.	Базовая библиотека Rust (`ndarray`), пользовательская реализация [36]
SIMD Acceleration	Последовательная обработка данных как узкое место.	Параллельная обработка с помощью SIMD-инструкций (AVX-512).	`packed_simd`, `trueno` [54]
Zen-Specific Tuning	Неиспользование улучшений архитектуры (IPC, память).	Адаптация алгоритмов (например, более сложные аппроксимации) под высокую пропускную способность памяти Zen 5.	Базовая библиотека Rust, пользовательская реализация [332]

Этот комплексный подход к аппаратной оптимизации позволит Рэто v10 не просто конкурировать с Adam, а стать его превосходством именно на оборудовании AMD Zen, где его специализированные функции будут работать в полную силу.

Механизмы Управления Памятью и Энергоэффективности

В условиях ограниченной оперативной памяти, характерной для обучения на CPU, эффективное управление ресурсами становится не менее важным, чем сама формула оптимизации. Рэто v10 будет разработан с учетом двух

ключевых аспектов управления ресурсами: минимизация накладных расходов на выделение и освобождение памяти и обеспечение стабильной работы без перегрева процессора. Эти механизмы будут реализованы через специализированные аллокаторы и стратегии энергосбережения.

Первый механизм — это переход от стандартных аллокаторов к специализированным. В Rust, как и в C++, стандартный менеджер памяти (часто jemalloc или mimalloc) предназначен для универсальных задач, но в контексте нейросетей он может быть неоптимальным. Многократные вызовы `malloc`/`free` для создания и уничтожения небольших буферов (например, для активаций, градиентов или моментов на каждом шаге) несет в себе значительные накладные расходы, связанные с поиском подходящего блока памяти, блокировками в многопоточной среде и фрагментацией [87] [208]. Для Рэто v10 будет внедрен механизм пулов аллокаторов и, в особенности, аллокаторов типа «bump allocator». Библиотека `bumpalo` является примером такого подхода [319]. Аллокатор типа «bump» работает по принципу «толкай», то есть он выделяет большой блок памяти («арену») и затем просто перемещает указатель вперед для выделения новых объектов. Освобождение всей памяти, выделенной внутри арены, происходит за константное время — всего лишь сбросом указателя. Это идеальный подход для цикла обучения, где временные буферы создаются и уничтожаются на каждой итерации. Используя `bumpalo` для выделения градиентов, активаций и промежуточных вычислений, мы можем избавиться от почти всех накладных расходов на аллокацию памяти, что приведет к заметному приросту производительности [129]. Для более сложных случаев, где память нужно освобождать постепенно, могут быть использованы пулы объектов, которые кэшируют освобожденные блоки памяти для их повторного использования [173].

Второй механизм — это управление памятью модели и состояний оптимизатора. Как уже упоминалось, основной проблемой для помещаемых в ОЗУ моделей является объем, занимаемый состояниями Adam (m_t и v_t). Рэто v10 будет использовать INT8 квантование для хранения этих состояний, что сократит их размер в 4 раза по сравнению с FP32 [233]. Это позволит поместить в ОЗУ модели с вдвое большим количеством параметров или оставить больше места для кэша активаций. Для компенсации потерь точности, возникающих при квантовании, будет реализован механизм обратной связи об ошибке. Этот механизм сохраняет округленную ошибку от предыдущего шага и добавляет ее к следующему градиенту перед квантованием, что позволяет контролировать

накопление ошибки и поддерживать стабильность обучения [199]. Этот подход был успешно применен в PyTorch для 8-битных оптимизаторов [199].

Третий аспект — это энергоэффективность и термический контроль. Цель исследования — отказ от GPU-специфичных решений в пользу чисто CPU-оптимизированных подходов, обеспечивающих стабильную работу без перегрузки системы. CPU, особенно на ноутбуках или в компактных системах, имеет ограниченный TDP (потребляемая тепловая мощность). Высокая нагрузка на ЦП приводит к повышению температуры, что вызывает троттлинг — автоматическое снижение тактовой частоты для предотвращения перегрева. Это напрямую снижает производительность. Рэто v10 будет спроектирован с учетом этого ограничения.

Для управления температурой и энергопотреблением будет реализован ряд стратегий. Во-первых, это оптимизация самого кода для максимальной производительности на цикл (IPC). Использование SIMD, тайлинг и NUMA-согласованное выделение памяти не только ускоряет вычисления, но и делает их более энергоэффективными, так как процессор выполняет ту же работу за меньшее время и может проводить больше времени в спящем режиме. Во-вторых, в рамках фреймворка Ryza будет реализован механизм динамической регулировки частоты. Хотя прямое управление частотой процессора сложно в пользовательском пространстве, можно использовать существующие API для запроса более высоких или более низких частот. Например, если система находится на питании от сети, можно позволить процессору работать на более высоких частотах. Если же питание осуществляется от аккумулятора, можно установить более низкую целевую производительность, чтобы сэкономить энергию. Библиотеки, такие как `zenpower` для Linux, позволяют отслеживать температуру и напряжение процессоров AMD Zen, что может быть использовано для создания более сложных систем реального времени [63].

В-третьих, будет рассмотрена возможность использования cgroups v2 для ограничения ресурсов процесса обучения. Cgroups позволяют группировать процессы и устанавливать для них жесткие лимиты на потребление CPU и памяти [90] [224]. Это позволяет гарантировать, что процесс обучения не сможет "захламить" всю систему и вызвать зависание других приложений. Например, можно ограничить процесс до использования определенного процента CPU (например, 80%), оставляя оставшийся 20% свободным для системных задач и для повышения частоты в случае необходимости. Это обеспечивает более предсказуемую и стабильную работу системы в долгосрочной перспективе [223].

Наконец, для мониторинга энергопотребления и производительности будут использоваться инструменты командной строки, такие как `perf` в Linux. `perf` может собирать статистику по событиям производительности, включая количество циклов процессора, промахи в кэше и энергопотребление (при наличии соответствующих сенсоров). Это позволит проводить детальный анализ производительности Рэто v10 и находить дальнейшие точки для оптимизации [33]. Таким образом, управление памятью и энергоэффективность в Рэто v10 — это не второстепенные задачи, а integralная часть архитектуры, которая обеспечивает его жизнеспособность и эффективность в реальных условиях ограниченных ресурсов.

Механизм	Проблема	Решение в Рэто **v10**	Библиотеки/Инструменты
Выделение памяти	Накладные расходы на `malloc`/`free` и фрагментация.	Использование аллокатора типа «bump» для временных буферов.	`bumpalo`, `memory-pool` [319]
Хранение состояний	Большой объем памяти, занимаемый моментами Adam.	INT8 квантование состояний с компенсацией ошибки.	Собственная реализация
Энергоэффективность	Высокое энергопотребление и перегрев CPU.	Оптимизация кода (SIMD, тайлинг), динамическая регулировка частоты.	`perf`, `zenpower` [63]
Стабильность системы	Процесс обучения может исчерпать все ресурсы CPU/памяти.	Использование cgroups v2 для ограничения ресурсов.	Системные утилиты Linux (`cgcreate`, `cgexec`) [224]

Эти механизмы создают надежный и эффективный фундамент для Рэто v10, позволяя ему не только быть математически совершенным, но и практически применимым в реальных условиях ограниченных ресурсов.

Проектирование и Реализация Рэто **v10:** От Концепции к Коду

Проектирование и реализация оптимизатора Рэто v10 — это синтез всех вышеописанных концепций в единую, работоспособную систему. Этот процесс начинается с определения архитектуры основной структуры `RetoV10`, которая будет содержать все необходимые состояния и гиперпараметры. Затем следует реализация основного цикла обновления, который будет инкапсулировать сложную логику, включающую ядро Adam, математические улучшения и аппаратные оптимизации. Наконец, весь код будет интегрирован в существующий проект Ryza, заменяя старые версии оптимизатора.

Первым шагом является определение структуры `RetoV10`. Она должна быть достаточно гибкой, чтобы поддерживать различные режимы работы (например, с естественным градиентом или без него), но при этом эффективной с точки зрения использования памяти. Структура будет включать в себя все необходимые состояния Adam: `m` (первый момент), `v` (второй момент) и `step` (текущий шаг). Кроме того, она будет содержать гиперпараметры Adam (`beta1`, `beta2`, `epsilon`, `lr`) и AdamW (`weight_decay`). Особое внимание будет уделено состояниям, связанным с памятью и спецификой ЦП. Будет добавлено поле для `error_buffer` (буфер ошибок квантования), которое будет активироваться только при использовании INT8. Также будут поля для хранения параметров, связанных с геометрическими улучшениями, например, низкоранговые матрицы для аппроксимации информации Фишера.

```
use ndarray::{Array1, Array2};

/// Рэто v10: Оптимизатор для CPU с поддержкой естественных градиентов и I
pub struct RetoV10 {
    // === Состояния Adam ===
    pub m: Array1<f32>,
    pub v: Array1<f32>,

    // === Состояния для INT8 квантования ===
    pub error_buffer: Option<Array1<f32>>,

    // === Состояния для Natural Gradient (K-FAC like) ===
    // Здесь будут храниться низкоранговые аппроксимации матриц,
    // необходимых для вычисления естественного градиента.
    // Например, низкоранговые факторы матрицы Гессиана или Фишера.
    pub low_rank_factors: Option<(Array2<f32>, Array2<f32>)>, // (U, V)

    // === Гиперпараметры ===
    pub lr: f32,
    pub beta1: f32,
    pub beta2: f32,
    pub epsilon: f32,
    pub weight_decay: f32,
    pub entropic_reg_weight: f32, // Коэффициент энтропийной регуляризации

    // === Счетчик шагов и другие флаги ===
    pub step: u64,
```

```
    pub use_int8: bool,
    pub use_natural_gradient: bool,
    pub use_entropic_reg: bool,
}
```

Основной метод `step` будет реализовывать сложный алгоритм обновления. Он будет состоять из нескольких последовательных шагов. На первом шаге будут вычислены градиенты. Затем, если включен флаг `use_entropic_reg`, будет вычислен градиент энтропийного регуляризатора и добавлен к основному градиенту. После этого, если `use_int8` равно true, градиент будет квантован с использованием `error_buffer`. Далее, как и в Adam, будут обновлены моменты `m` и `v`. После этого, если `use_natural_gradient` равно true, будет вычислена поправка, основанная на низкоранговых аппроксимациях, и она будет добавлена к скорректированному градиенту. Наконец, будет применена коррекция смещения, и окончательное обновление весов будет выполнено с учетом декуплированного затухания весов.

```
impl RetoV10 {
    pub fn step(&mut self, weights: &mut Array1<f32>, gradients: &mut Arra
        self.step += 1;

        // 1. Entropic Regularization Gradient (если включено)
        let mut total_gradients = gradients.clone();
        if self.use_entropic_reg {
            let entropy_grad = self.compute_entropy_gradient(weights);
            total_gradients += &entropy_grad;
        }

        // 2. INT8 Quantization with Error Feedback (если включено)
        let quantized_grad = if self.use_int8 {
            self.quantize_with_feedback(&mut total_gradients)
        } else {
            total_gradients.clone()
        };

        // 3. Update Adam moments
        for i in 0..self.m.len() {
            self.m[i] = self.beta1 <em> self.m[i] + (1.0 - self.beta1) </e
            self.v[i] = self.beta2 <em> self.v[i] + (1.0 - self.beta2) </e
        }
```

```
        // 4. Bias Correction
        let bias_correction_m = 1.0 - self.beta1.powi(self.step as i32);
        let bias_correction_v = 1.0 - self.beta2.powi(self.step as i32);
        let corrected_m = &self.m / bias_correction_m;
        let corrected_v = &self.v / bias_correction_v;

        // 5. Natural Gradient Correction (если включено)
        let natural_grad_correction = if self.use_natural_gradient {
            self.compute_natural_gradient_correction(&corrected_m, &correc
        } else {
            Array1::ones(corrected_m.len())
        };

        // 6. Final update
        for i in 0..weights.len() {
            let update_step = self.lr * corrected_m[i] / (corrected_v[i].s
            let final_update = update_step * natural_grad_correction[i];

            // Decoupled Weight Decay
            weights[i] = weights[i] <em> (1.0 - self.lr </em> self.weight_
        }
    }

    // ... остальные методы: compute_entropy_gradient, quantize_with_feedb
}
```

Реализация метода `compute_natural_gradient_correction` будет самой сложной. Он должен будет использовать низкоранговые факторы, хранящиеся в `low_rank_factors`, для аппроксимации действия матрицы, обратной к матрице Фишера (или Гессиана). Это можно сделать, используя лемму о матрице-шершавке или другие численные методы для работы с низкоранговыми матрицами. Например, если $G = I + U\Sigma V^T$, то $G^{-1} \approx I - U(I + \Sigma V^T U)^{-1}\Sigma V^T$. Это и будет наша поправка.

```
// Пример псевдокода для compute_natural_gradient_correction
fn compute_natural_gradient_correction(&self, grad: &Array1<f32>) -> Array
    if let Some((ref u, ref v)) = self.low_rank_factors {
        // Применяем низкоранговую поправку к градиенту
        // Это сильно упрощенная версия
```

```
        let uv_prod = v.dot(u.t().dot(grad));
        let correction = grad - u.t().dot(&(uv_prod / (1.0 + uv_prod)));
        correction
    } else {
        Array1::ones(grad.len())
    }
}
```

Этот пример показывает концепцию, но реальная реализация потребует более тщательной работы с линейной алгеброй. Для этого можно использовать библиотеку `ndarray-linalg`.

Наконец, весь этот код будет интегрирован в проект Ryza. Старая версия `reto_optimizer` будет заменена новой реализацией `RetoV10`. В файле `main.rs` или в основном файле проекта Ryza нужно будет настроить гиперпараметры Рэто v10, выбрав режимы работы (например, сначала протестировать без естественных градиентов, а затем добавить их).

```
// Пример инициализации в Ryza
let n_params = model.total_parameters();
let mut optimizer = RetoV10::new(
    n_params,
    lr: 0.001,
    use_int8: true,
    use_natural_gradient: true,
    use_entropic_reg: true,
    entropic_reg_weight: 0.01,
);
```

Для сборки и запуска проекта потребуется убедиться, что в `Cargo.toml` правильно указаны зависимости, включая `ndarray`, `ndarray-linalg` и, возможно, `numa-rs` для NUMA-функциональности.

```
[dependencies]
ndarray = { version = "0.15", features = ["serde", "linalg"] }
rand = "0.8"
serde = { version = "1.0", features = ["derive"] }
numa-rs = "0.1" # Для NUMA-операций
bumpalo = "3.13" # Для bump allocator
```

Этот пошаговый процесс — от проектирования структуры до реализации логики и интеграции в проект — позволит создать полнофункциональную и мощную версию оптимизатора Рэто v10. Каждый компонент будет тщательно продуман для достижения максимальной производительности на CPU, эффективного использования памяти и интеграции передовых математических идей.

Эмпирическая Валидация и Сравнительный Анализ Производительности

Создание оптимизатора Рэто v10 — это лишь половина дела. Чтобы доказать его превосходство над Adam, необходимо провести строгую эмпирическую валидацию на стандартизированных тестовых задачах и в реальных сценариях обучения. Этот этап валидации будет состоять из трех ключевых частей: выбора репрезентативных бенчмарков, измерения релевантных метрик производительности и проведения сравнительного анализа полученных результатов.

Выбор правильных бенчмарков критически важен для оценки сильных и слабых сторон оптимизатора. Мы будем использовать три классических теста, которые хорошо известны в мире оптимизации: 1. Функция Розенброка: Это "бананообразная" функция, являющаяся стандартным тестом для оптимизаторов из-за своей узкой, изогнутой долины, где находится глобальный минимум [68] [270]. Она проверяет способность оптимизатора эффективно двигаться по сложным, плохо условленным поверхностям потерь. Adam обычно показывает хорошие результаты на этой задаче, поэтому Рэто v10 должен либо сравняться, либо превзойти его [237]. 2. Искаженная квадратичная функция: Эта задача состоит из суммы квадратов по каждой переменной, но с сильно различающимися коэффициентами (например, $f(x) = \sum_i \lambda_i x_i^2$, где λ_i меняются на несколько порядков). Она напрямую тестирует адаптивность скоростей обучения, так как для каждой переменной требуется совершенно разный шаг. Именно здесь NLMS-подобные подходы и естественные градиенты могут дать преимущество [1] [3]. 3. Шумная функция: Для этой задачи к идеальному градиенту добавляется случайный шум. Это проверяет устойчивость оптимизатора к неточным или "грязным" оценкам градиента, что типично для стохастического градиентного спуска. Adam и его варианты должны быть более устойчивыми к

шуму благодаря сглаживанию моментами, но Рэто v10 с его INT8 квантованием и энтропийной регуляризацией должен показать интересные результаты [340].

Для каждой из этих задач мы проведем серию экспериментов, сравнивая производительность Рэто v10 с несколькими вариантами Adam:

- **Adam (стандартный):** Реализация Adam из фреймворка.
- **AdamW:** Версия Adam с декуплированным затуханием весов, которая считается современным стандартом [378].
- Рэто **v9.0 (базовый):** Предыдущая версия, для оценки прогресса.

Измеряемые метрики будут включать:

- Скорость сходимости: Количество итераций (или время) для достижения определенного порога ошибки (например, $f(x) < 10^{-6}$).
- Качество решения: Финальное значение функции потерь после фиксированного числа итераций.
- Потребление памяти: Максимальный объем оперативной памяти, потребляемый процессом обучения. Это будет особенно важно для сравнения версий с INT8 квантованием.
- Энергоэффективность: Измерение энергопотребления с помощью инструментов вроде `rapl` или `powerstat` в Linux. Это позволит оценить, какой из оптимизаторов является более "зеленым".
- **CPU-**использование и кэш-промахи: Сбор статистики производительности с помощью `perf stat` для анализа, насколько эффективно код использует CPU, и как часто происходят промахи в кэше.

После сбора данных будет проведен сравнительный анализ. Результаты будут представлены в виде таблиц и графиков, показывающих сходимость для каждой задачи. Например, для функции Розенброка график будет показывать логарифм ошибки в зависимости от времени. Для искаженной квадратичной функции мы сравним финальные значения ошибки для каждого оптимизатора. Если Рэто v10 покажет сходимость быстрее, чем AdamW, на искаженной квадратичной задаче, это станет сильным доказательством эффективности его геометрических улучшений. Если он достигнет той же ошибки, но с меньшим потреблением памяти благодаря INT8, это будет доказательством успеха в области управления ресурсами.

Важно отметить, что результаты не всегда будут однозначными. AdamW может по-прежнему лидировать на задаче Розенброка, так как его адаптивность

хорошо подходит для таких поверхностей [8]. В этом случае выводом будет не "Рэто проигрывает", а "На данной задаче AdamW оказывается более эффективным". Это ценная информация, которая поможет понять сильные стороны каждого алгоритма. Возможно, Рэто v10 окажется лучше на задачах с высокой размерностью или с очень большим разбросом масштабов, где его низкоранговые аппроксимации дадут ощутимый прирост.

Метрика	Описание	Инструменты
Скорость сходимости	Время или количество итераций до достижения цели.	Стандартное время выполнения, логирование.
Качество решения	Финальное значение функции потерь.	Логирование значения функции потерь.
Потребление памяти	Максимальный объем ОЗУ, используемый процессом.	`valgrind --tool=massif`, `/usr/bin/time -v`.
Энергоэффективность	Общее энергопотребление за время обучения.	`rapl` сенсоры, `powerstat`.
CPU-использование	Процент времени, когда CPU занят.	`top`, `htop`.
Кэш-промахи	Количество промахов в разных уровнях кэша.	`perf stat -e cache-misses,instructions`.

Этот строгий и всесторонний подход к валидации позволит не просто заявить о превосходстве Рэто v10, а доказать его на основе фактических данных. Он выявит не только сильные стороны новой архитектуры, но и области, требующие дальнейшей доработки, обеспечивая научно обоснованный путь для дальнейшего развития оптимизатора.

Справка

1. Poorly conditioned quadratic programming with "simple" linear ... https://or.stackexchange.com/questions/9265/poorly-conditioned-quadratic-programming-with-simple-linear-constraints
2. SWAN: Preprocessing SGD Enables Adam-Level Performance On LLM ... https://arxiv.org/html/2412.13148v1
3. Taming ill-conditioned problems in machine learning: algorithms, ... https://web.stanford.edu/~udell/doc/rathore26_thesis.pdf
4. A Quadratic Programming Bibliography https://optimization-online.org/wp-content/uploads/2001/02/285.pdf

5. Kronecker Approximation-Domination Gradient Preconditioned Stochastic ... https://proceedings.mlr.press/v216/mei23a/mei23a.pdf
6. HIFIR: Hybrid Incomplete Factorization with Iterative ... https://dl.acm.org/doi/10.1145/3536165
7. OSQP: an operator splitting solver for quadratic programs | Request PDF https://www.researchgate.net/publication/321241585_OSQP_An_Operator_Splitting_Solver_for_Quadratic_Programs
8. Adam Optimizer: Adaptive Learning Rates Explained - Interactive https://mbrenndoerfer.com/writing/adam-optimizer-deep-learning
9. EPANG-Gen: A Curvature-Aware Optimizer with Uncertainty ... https://www.preprints.org/manuscript/202603.0692
10. Memory Optimization Overview — torchtune 0.5 documentation https://meta-pytorch.org/torchtune/0.5/tutorials/memory_optimizations.html
11. [D] Is there a theoretically justified reason for choosing an optimizer for ... https://www.reddit.com/r/MachineLearning/comments/j302g8/d_is_there_a_theoretically_justified_reason_for/
12. Optimizers in Machine Learning and AI: A Comprehensive Overview https://medium.com/@anshm18111996/comprehensive-overview-optimizers-in-machine-learning-and-ai-57a2b0fbcc79
13. Deconstructing What Makes a Good Optimizer for Language Models https://arxiv.org/html/2407.07972v1
14. Optimizing Memory Usage when Training Deep Neural Networks https://theses.hal.science/tel-04890912/file/ZHAO_XUNYI_2024.pdf
15. Lightweight Optimization Techniques for Neural Network Training Using ... https://www.authorea.com/doi/10.22541/au.176549085.50277488
16. Introducing Adam Accumulation: A New Optimizer for Deep Learning https://www.linkedin.com/posts/pavankumargedala_adama-deeplearningoptimization-gpuacceleration-activity-7394932090766745600-2wpu
17. Gentle Introduction to the Adam Optimization Algorithm for Deep ... https://machinelearningmastery.com/adam-optimization-algorithm-for-deep-learning/
18. Choosing the Right Optimizer for Your Deep Learning Problem https://genmind.ch/posts/Choosing-the-Right-Optimizer-for-Your-Deep-Learning-Problem/
19. AdamW Optimization in Deep Learning https://www.emergentmind.com/topics/adamw-optimization
20. Adam (Adaptive Moments) I https://www.csie.ntu.edu.tw/~cjlin/courses/optdl2025/slides/sg_sg2.pdf

21. The 6 Optimization Algorithms: How AI Learns to Learn 10× Faster with 50% ... https://pub.towardsai.net/the-6-optimization-algorithms-how-ai-learns-to-learn-10-faster-with-50-less-memory-1709c9168d3a
22. Adam: Adaptive Moment Estimation - optimī https://optimi.benjaminwarner.dev/optimizers/adam/
23. On the Overlooked Pitfalls of Weight Decay and How to Mitigate Them https://arxiv.org/html/2011.11152v6
24. Adaptive Learning Rates - Adam, AdaGrad, RMSprop https://www.tensortonic.com/ml-math/optimization/adaptive-rates
25. Adaptive optimizers address uneven feature dynamics by changing per ... https://www.facebook.com/mql5.community/posts/adaptive-optimizers-address-uneven-feature-dynamics-by-changing-per-parameter-le/1362374466001555/
26. uhub/awesome-rust https://github.com/uhub/awesome-rust
27. HyLo: A Hybrid Low-Rank Natural Gradient Descent Method https://www.cs.toronto.edu/~mmehride/papers/HyLo.pdf
28. Improving Gradient Descent for Better Deep Learning with ... https://machine-learning-made-simple.medium.com/improving-gradient-descent-for-better-deep-learning-with-natural-gradients-327e5faa836a
29. Optimizing Neural Networks with Kronecker-factored Approximate ... https://arxiv.org/abs/1503.05671
30. The K-FAC method for neural network optimization https://app.icerm.brown.edu/materials/Slides/sp-s19-w1/The_K-FAC_method_for_neural_network_optimization_]_James_Martens,_DeepMind.pdf
31. Kronecker-Factored Approx. Curvature (K-FAC) https://www.emergentmind.com/topics/kronecker-factored-approximate-curvature-k-fac-cfb12415-7bdc-42ec-b0f2-bb04c447624a
32. A Kronecker-factored approximate Fisher matrix for convolution layers https://proceedings.mlr.press/v48/grosse16.html
33. dial9-tokio-telemetry - crates.io: Rust Package Registry https://crates.io/crates/dial9-tokio-telemetry
34. An Improved BGE-Adam Optimization Algorithm Based on ... https://www.mdpi.com/2073-8994/16/5/623
35. BDS-Adam optimizer integrating adaptive variance ... - PMC https://pmc.ncbi.nlm.nih.gov/articles/PMC12546589/
36. locality, access, allocate, cache lines - tutorials https://users.rust-lang.org/t/tutorial-on-rust-memory-performance-latency-locality-access-allocate-cache-lines/130718

37. Does Vec> hurt CPU cache locality? : r/rust https://www.reddit.com/r/rust/comments/10w2odm/does_vecoptiont_hurt_cpu_cache_locality/
38. Understanding cache-locality with an example (in Rust) https://www.youtube.com/watch?v=n-AsCx0142Y
39. Rust is faster than C, even before I added SIMD - Blargh https://blog.habets.se/2024/04/Rust-is-faster-than-C.html
40. Build with Naz - Ep 21 : High Performance Flat 2D Arrays in ... https://www.youtube.com/watch?v=tIrSvJFRxAg
41. Running Gaming Workloads through AMD's Zen 5 https://chipsandcheese.com/p/running-gaming-workloads-through
42. Build High-Performance Flat 2D Arrays in Rust (SIMD, L1 ... https://developerlife.com/2026/07/14/build-high-performance-flat-2d-arrays-in-rust/
43. Building Distributed Caches in Rust: 1000x Faster ... https://medium.com/@yashbatra11111/building-distributed-caches-in-rust-1000x-faster-performance-than-youd-expect-7fa1fcf34354
44. Introduction - Rust SIMD Performance Guide https://rust-lang.github.io/packed_simd/perf-guide/introduction.html
45. Exploring NUMA Optimization with Real World Scenarios https://linuxgd.medium.com/exploring-numa-optimization-with-real-world-scenarios-d75a37e2c10d
46. MAO: Machine learning approach for NUMA optimization in ... https://arxiv.org/html/2411.01460v1
47. How to Optimize NUMA Memory Allocation for Multi-Socket ... https://oneuptime.com/blog/post/2026-03-04-optimize-numa-memory-allocation-for-multi-socket-servers/view
48. NUMA-aware Memory Allocation https://www.gridgain.com/docs/tutorials/numa-allocation/numa-allocation-tutorial
49. numactl(8) - Linux manual page https://man7.org/linux/man-pages/man8/numactl.8.html
50. how do numactl & perf change memory placement policy of ... https://stackoverflow.com/questions/62604334/how-do-numactl-perf-change-memory-placement-policy-of-child-processes
51. Chapter 9. NUMA | Virtualization Tuning and Optimization ... https://docs.redhat.com/en/documentation/red_hat_enterprise_linux/7/html/virtualization_tuning_and_optimization_guide/chap-virtualization_tuning_optimization_guide-numa
52. What is NUMA? — The Linux Kernel documentation https://www.kernel.org/doc/html/v5.7/vm/numa.html

53. Memory Bandwidth - Linux Kernel Internals https://kernel-internals.org/mm/memory-bandwidth/
54. GitHub - paiml/trueno: MOVED → https ... https://github.com/paiml/trueno
55. saforem2/awesome-stars: A curated list of my ... https://github.com/saforem2/awesome-stars
56. microgpt · GitHub https://gist.github.com/karpathy/8627fe009c40f57531cb18360106ce95?permalink_comment_id=5997071
57. awesome-stars/README.md at master https://github.com/saforem2/awesome-stars/blob/master/README.md
58. parselmouth/files/compressed_mapping.json at main https://github.com/prefix-dev/parselmouth/blob/main/files/compressed_mapping.json
59. awesome-stars/README.md at master · jiegec ... https://github.com/jiegec/awesome-stars/blob/master/README.md?96NiWNYc1FZ=c57K6cmqIgTC
60. tsirysndr/my-awesome-stars https://github.com/tsirysndr/my-awesome-stars
61. Calibration data used for Qwen3, includes original work ... https://gist.github.com/bartowski1182/f003237f2e8612278a6d01622af1cb6f
62. uhub/awesome-css https://github.com/uhub/awesome-css
63. stars/README.md at main · Thomashighbaugh/stars https://github.com/Thomashighbaugh/stars/blob/main/README.md
64. AVX-512: First Impressions on Performance and ... https://news.ycombinator.com/item?id=46610800
65. Linux_6.18 https://kernelnewbies.org/Linux_6.18
66. The kernel's command-line parameters https://docs.kernel.org/admin-guide/kernel-parameters.html
67. The Complete Computing Handbooks Parallel and Distributed ... https://simplifycpp.org/books/SEH_Book13_Parallel_and_Distributed_Computing.pdf
68. OPTIMIZATION ALGORITHMS FOR TRAINING NEURAL ... https://arxiv.org/html/2512.18373v1
69. First-order optimization algorithms: state of the art ... https://link.springer.com/article/10.1007/s00521-026-12014-1
70. Track: Poster Session 2 East https://neurips.cc/virtual/2024/session/108364
71. CMACGSA: Improved Gravitational Search Algorithm ... https://ieeexplore.ieee.org/iel8/6287639/10820123/10855996.pdf
72. Offline Model-Based Optimization Comprehensive Review https://www.tdcommons.org/cgi/viewcontent.cgi?article=9245&context=dpubs_series

73. ACCELERATING PRACTICAL ENGINEERING DESIGN ... https://dspace.mit.edu/bitstream/handle/1721.1/163228/ICAT-2024-06-Peter%20D.-Sharpe_Accelerating%20Practical%20Engineering%20Design%20Optimization%20with%20Computational%20Graph%20Transformations.pdf?sequence=1&isAllowed=y
74. Optimization Toolbox User's Guide https://www.mathworks.com/help/pdf_doc/optim/optim.pdf
75. Practical Augmented Lagrangian Methods for Constrained ... https://repositorio.usp.br/directbitstream/c1c8bd6c-0663-4fe3-9c09-7552e1ce14db/2482622.pdf
76. IFAC WC 2026 Program | Wednesday August 26, 2026 https://ifac.papercept.net/conferences/conferences/IFAC26/program/IFAC26_ContentListWeb_3.html
77. What Are The Best Settings for an AMD Ryzen CPU? https://www.youtube.com/watch?v=N60M36PRHsY&vl=en
78. AMD Zen2 Optimization Guide (Discoverer CPU partition) https://docs.discoverer.bg/zen2.html
79. tuning guide - amd epyc 9004 https://docs.amd.com/api/khub/documents/NgOfoW49HKdzztTeLBbekA/content
80. AMD Zen 6 architecture to improve 1% lows with ... https://www.reddit.com/r/Amd/comments/1vcjp8d/amd_zen_6_architecture_to_improve_1_lows_with/
81. AMD Zen 6 architecture to improve 1% lows with ... https://videocardz.com/newz/amd-zen-6-architecture-to-improve-1-lows-with-performance-priority-controls
82. Tech Explainer: What is the AMD “Zen” core architecture? https://www.performance-intensive-computing.com/objectives/tech-explainer-what-is-the-amd-zen-core-architecture
83. How to Overclock your AMD CPU and Improve Performance ... https://www.youtube.com/watch?v=557p0n7RQ2c
84. AMD RYZEN PROCESSOR SOFTWARE OPTIMIZATION https://gpuopen.com/download/GDC2024_AMD_Ryzen_Processor_Software_Optimization.pdf
85. Software Optimization Guide for the AMD Zen4 ... https://www.numberworld.org/blogs/2024_8_7_zen5_avx512_teardown/57647_zen4_sog.pdf
86. Rust: Notes on memory management https://monicaspisar.com/posts/rust-memory-management/
87. Stochastic streams: Dynamic memory allocations in bare ... https://www.youtube.com/watch?v=W8M6wvZLMoQ
88. What are some tips that you follow to keep RUST program ... https://www.reddit.com/r/rust/comments/1jirmhy/what_are_some_tips_that_you_follow_to_keep_rust/

89. OOP 2024: Rust Memory Management Introduction https://www.youtube.com/watch?v=Vj1MuTTplD4
90. Chapter 26. Configuring resource management by using ... https://docs.redhat.com/en/documentation/red_hat_enterprise_linux/8/html/managing_monitoring_and_updating_the_kernel/assembly_configuring-resource-management-using-systemd_managing-monitoring-and-updating-the-kernel
91. Memory Allocations in Rust https://dev.to/gritmax/memory-allocations-in-rust-3m7l
92. How to Optimize Rust Memory Usage and Prevent ... https://oneuptime.com/blog/post/2026-01-07-rust-memory-optimization/view
93. Memory Management In Rust https://medium.com/@notcodesid/memory-management-in-rust-e835ea663784
94. AMD Zen Deep Neural Network (ZenDNN) https://www.amd.com/en/developer/zendnn.html
95. Low-Rank Gradient Descent for Memory-Efficient Training of Deep In ... https://dl.acm.org/doi/full/10.1145/3577214
96. Compute Efficient Low-Memory LLM Training with Structured Sparse ... https://aclanthology.org/2024.emnlp-main.835.pdf
97. NeuZip: Memory-Efficient Training and Inference with Dynamic ... https://arxiv.org/html/2410.20650v1
98. Compute Efficient Low-Memory LLM Training with Structured Sparse ... https://openreview.net/forum?id=HJezss5JIG
99. COMET: A Novel Memory-Efficient Deep Learning Training Framework ... https://www.vldb.org/pvldb/vol15/p886-tao.pdf
100. Entropic Regularization: Theory & Applications https://www.emergentmind.com/topics/entropic-regularization
101. Convergence rate of Tsallis entropic regularized optimal ... https://arxiv.org/html/2304.06616v2
102. Enforcing KL Regularization in General Tsallis Entropy ... https://www.researchgate.net/publication/360668074_Enforcing_KL_Regularization_in_General_Tsallis_Entropy_Reinforcement_Learning_via_Advantage_Learning
103. Entropy-Regularized Federated Optimization for Non-IID ... https://www.mdpi.com/1999-4893/18/8/455
104. An explicit analysis of the entropic penalty in linear programming https://proceedings.mlr.press/v75/weed18a/weed18a.pdf
105. An elementary introduction to entropic regularization and ... https://hal.science/hal-02303456v1/document

106. Gefen: Optimized Stochastic Optimizer https://arxiv.org/html/2606.13894v1
107. AdaMuon: Adaptive Muon Optimizer https://arxiv.org/html/2507.11005v3
108. Adapprox: Adaptive Approximation in Adam Optimization ... https://arxiv.org/pdf/2403.14958
109. Adaptive Memory Momentum via a Model-Based ... https://arxiv.org/pdf/2510.04988
110. Adam: A Method for Stochastic Optimization https://arxiv.org/pdf/1412.6980
111. Benchmarking Optimizers for Large Language Model ... https://arxiv.org/html/2509.01440v1
112. Amos: AN ADAM-STYLE OPTIMIZER WITH ADAPTIVE ... https://arxiv.org/pdf/2210.11693
113. Practical Efficiency of Muon for Pretraining https://arxiv.org/html/2505.02222v1
114. [1904.09237] On the Convergence of Adam and Beyond https://arxiv.org/abs/1904.09237
115. AMSgrad Variant (Adam) - Computer Vision Wiki - CloudFactory https://wiki.cloudfactory.com/docs/mp-wiki/solvers-optimizers/adam/amsgrad-variant-adam
116. Adam Variants: Adamax & Nadam https://apxml.com/courses/deep-learning-regularization-optimization/chapter-6-adaptive-optimizers/adam-variants-adamax-nadam
117. A Framework and Benchmark for Deep Batch Active ... https://jmlr.org/papers/volume24/22-0937/22-0937.pdf
118. Enhanced beetle antennae search algorithm for complex and ... https://pmc.ncbi.nlm.nih.gov/articles/PMC9392993/
119. Non-asymptotic Convergence of Adam-type Reinforcement ... https://ojs.aaai.org/index.php/AAAI/article/view/17252
120. Constrained Discrete Black-Box Optimization using Mixed ... https://proceedings.mlr.press/v162/papalexopoulos22a/papalexopoulos22a.pdf
121. Comparative Analysis of Adam Optimizer and Its Variants ... https://medium.com/@astitwa15108824/comparative-analysis-of-adam-optimizer-and-its-variants-adamax-rmsprop-and-adagrad-addressing-02889adc8735
122. What is Adam Optimizer? Deep Learning Guide https://www.ultralytics.com/glossary/adam-optimizer
123. Introduction To Adam Optimizer https://www.geeksforgeeks.org/deep-learning/adam-optimizer/
124. Adaptive Learning Rate - an overview https://www.sciencedirect.com/topics/computer-science/adaptive-learning-rate

125. Momentum, AdaGrad, RMSProp & Adam https://towardsdatascience.com/understanding-deep-learning-optimizers-momentum-adagrad-rmsprop-adam-e311e377e9c2/

126. An improved empirical fisher approximation for natural ... https://dl.acm.org/doi/10.5555/3737916.3742180

127. Notes on the Limitations of the Empirical Fisher Approximation https://www.inference.vc/on-empirical-fisher-information/

128. An Improved Empirical Fisher Approximation for Natural ... https://arxiv.org/html/2406.06420v1

129. Memory management in rustc - Rust Compiler ... https://rustc-dev-guide.rust-lang.org/memory.html

130. Reduced Memory Usage: Burn's Rusty Approach to Tensor ... https://burn.dev/blog/burn-rusty-approach-to-tensor-handling/

131. Mitigating the NUMA Effect on Task-Based Runtime Systems https://upcommons.upc.edu/bitstreams/747ad5f8-15c9-44fa-9397-8d1635ccb2ef/download

132. AMD Optimizes EPYC Memory with NUMA https://www.amd.com/content/dam/amd/en/documents/epyc-business-docs/white-papers/AMD-Optimizes-EPYC-Memory-With-NUMA.pdf

133. NUMA architecture quick refresher | by Huan Xu https://medium.com/@hxu296/numa-architecture-quick-refresher-08599f428298

134. Optimizing Linux for AMD EPYC™ 9005 Series Processors ... https://documentation.suse.com/sbp/tuning-performance/html/SBP-AMD-EPYC-5-SLES15SP6/index.html

135. Guide on how to stop Ryzen CPU's from overheating ... https://www.reddit.com/r/AMDHelp/comments/1etkpn7/guide_on_how_to_stop_ryzen_cpus_from/

136. AMD Ryzen™ Master Utility for Overclocking Control https://www.amd.com/en/products/software/ryzen-master.html

137. How To OPTIMIZE your RYZEN CPU For Gaming ... https://www.youtube.com/watch?v=j63qZS6L6VE

138. CPU performance optimization guide - part 2 https://gpuopen.com/learn/cpu-performance-guide/cpu-performance-guide-part2/

139. How To Boost AMD CPU's Performance https://www.cyberpowerpc.com/blog/how-to-boost-amd-cpus-performance/?srsltid=AfmBOooGPO8cI9FVDzWMwmGgtKNsWHUih_oU4-u_B9io1_Kd6ZAT2m8y

140. AMD Ryzen Master Guide: Safe CPU Tuning in 2026 https://box.co.uk/blog/amd-ryzen-master-guide

141. Enjoy Boost Your AMD Gaming: Core Tuning & ... https://my.sancarlo.co.uk/sancarlo-news/boost-your-amd-gaming-core-tuning-and-performance-guide-1764802822

142. Increase AMD Ryzen CPU performance and reduce its ... https://www.youtube.com/watch?v=yfCIZci06Uc

143. Benchmark Comparison · Issue #215 · rust-lang/portable- ... https://github.com/rust-lang/portable-simd/issues/215

144. Learnings from making things fast : r/rust https://www.reddit.com/r/rust/comments/1516ctn/learnings_from_making_things_fast/

145. Rust SIMD — a tutorial. SIMD in Rust | by BWinter https://medium.com/@bartekwinter3/rust-simd-a-tutorial-bb9826f98e81

146. Speeding up parallel SIMD simulation https://users.rust-lang.org/t/speeding-up-parallel-simd-simulation/62896

147. Rust Performance Optimization: Complete Guide 2026 https://reintech.io/blog/rust-performance-optimization-complete-guide-2026

148. Building High Performance Search Indexes in Rust with SIMD ... https://www.youtube.com/watch?v=t_ALyJ174gs

149. FCLK - UCLK - MCLK speeds and other info https://www.overclock.net/threads/fclk-uclk-mclk-speeds-and-other-info.1802441/

150. ASUS NUC 16 Pro Copilot+ PC | For Everyday OpenClaw https://www.asus.com/displays-desktops/nucs/nuc-mini-pcs/asus-nuc-16-pro/

151. Headset level spatial computing in a pair of glasses Link in bio ... https://www.instagram.com/reel/DZq8HjesdMp/

152. My game station is looking awesome now Computer specs https://www.facebook.com/groups/PcBuildersCommunity/permalink/3092341274382566/

153. Modules Karolina - Introduction - IT4Innovations https://docs.it4i.cz/en/docs/software/modules/modules-karolina

154. USCIS - H-1B Approved Petitioners Fiscal Year 2009 https://www.aila.org/aila-files/96380C61-8DF4-47C3-9767-0CD36E80F216/10042060.pdf

155. 736 kB https://huggingface.co/microsoft/cocolm-base/resolve/main/dict.txt?download=true

156. h-1b-fy09 counts-employers.csv https://www.visalawyerblog.com/files/2014/02/list-H1B-employers-2009.pdf

157. AMD presenta DGFS, una tecnología para comprimir mejor la ... https://www.instagram.com/reel/DYFMZrtFEQx/

158. Exploring Zen 5 and Zen 4 Microarchitectures: Dive into Op ... https://medium.com/@jason890418123/exploring-zen-5-and-zen-4-microarchitectures-dive-into-op-cache-branch-prediction-and-more-f9da2469fb5e

159. AMD BIOS Tuning Guide Impact For Boosting AI/ML ... https://www.phoronix.com/review/amd-epyc-ai-ml-tuning/3

160. CPU performance optimization guide - part 1 https://gpuopen.com/learn/cpu-performance-guide/cpu-performance-guide-part1/

161. AMD's MLPerf Training Debut: Optimizing LLM Fine-Tuning ... https://rocm.blogs.amd.com/artificial-intelligence/mlperf-training-v5.0/README.html

162. William Lam's Post https://www.linkedin.com/posts/lamwilliam_quick-tip-reducing-high-cpu-utilization-activity-7488984155763089408-fSfv

163. An Active-Set Algorithm for Convex Quadratic ... https://www.mdpi.com/2227-7390/13/9/1467

164. Accelerated optimization in deep learning with a proportional ... https://pmc.ncbi.nlm.nih.gov/articles/PMC11599775/

165. Optimizing the Optimizer for Physics-Informed Neural ... https://arxiv.org/html/2501.16371v6

166. Optimization (scipy.optimize) — SciPy v1.18.0 Manual https://docs.scipy.org/doc/scipy/tutorial/optimize.html

167. program-book.pdf https://ios2026.isye.gatech.edu/sites/default/files/2026-03/program-book.pdf

168. Model-Based Derivative-Free Optimization Methods and ... https://optimization-online.org/wp-content/uploads/2023/02/thesis.pdf

169. Theory Manual https://www.sandia.gov/app/uploads/sites/241/2023/03/Theory-6.16.0.pdf

170. An improved gorilla troops optimizer for enhanced parameter ... https://www.emerald.com/wje/article/doi/10.1108/WJE-10-2025-0670/1366804/An-improved-gorilla-troops-optimizer-for-enhanced

171. Tips on how to deal with dynamic memory allocations? : r/rust https://www.reddit.com/r/rust/comments/1o2xo7n/tips_on_how_to_deal_with_dynamic_memory/

172. Control the timing of dynamic allocation - help https://users.rust-lang.org/t/control-the-timing-of-dynamic-allocation/94775

173. Custom Allocators and RAII Patterns https://intellinesia.com/blog/rust-memory-management

174. Non-Lexical Lifetimes and Custom Allocators | by FAANG https://medium.com/@FAANG/advanced-memory-management-in-rust-non-lexical-lifetimes-and-custom-allocators-c32a277ffa4b

175. Mixture of Experts with Entropic Regularization for Data ... https://www.mdpi.com/1099-4300/21/2/190

176. amd/ZenDNN https://github.com/amd/ZenDNN

177. ZenDNN 5.0: Supercharge AI on AMD EPYC™ Server CPUs https://www.amd.com/en/developer/resources/technical-articles/zendnn-5-0-supercharge-ai-on-amd-epyc-server-cpus.html

178. SIMD programming in pure Rust https://news.ycombinator.com/item?id=46676861

179. Rust on CPU 3/4: ndarray, Gray-Scott and SIMD https://www.youtube.com/watch?v=DfJnR2-iGSY

180. SIMD programming in pure Rust https://kerkour.com/introduction-rust-simd

181. Concepts — ZeroEntropy https://zeroentropy.dev/concepts/

182. Enhancing Model loss improvement with Gradient Entropy ... https://pub.aimind.so/enhancing-model-loss-improvement-with-gradient-entropy-based-regularization-6dc4f96cb0ab

183. Fast Global Convergence of Natural Policy Gradient Methods ... https://users.ece.cmu.edu/~yuejiec/papers/EntropyNPG.pdf

184. An Improved Empirical Fisher Approximation for Natural ... https://www.researchgate.net/publication/381307668_An_Improved_Empirical_Fisher_Approximation_for_Natural_Gradient_Descent

185. Limitations of the Empirical Fisher Approximation for ... https://proceedings.nips.cc/paper/2019/file/46a558d97954d0692411c861cf78ef79-Paper.pdf

186. An Improvement of Adam Based on a Cyclic Exponential ... https://www.mdpi.com/2079-9292/13/9/1778

187. Comparison of Optimization Algorithms: SGD, Adagrad, ... https://aiml.com/comparing-optimization-algorithms-in-deep-learning-sgd-adam-and-more/

188. A Comparative Study of Optimization Techniques in Deep ... https://indjst.org/download-article.php?Article_Unique_Id=INDJST13981&Full_Text_Pdf_Download=True

189. Optimizers — DeepSpeed 0.19.3 documentation https://deepspeed.readthedocs.io/en/latest/optimizers.html

190. A modified Adam algorithm for deep neural network optimization https://link.springer.com/article/10.1007/s00521-023-08568-z

191. Adam – Optimization in Machine Learning https://wordpress.cs.vt.edu/optml/2018/03/13/adam/

192. Adam- The most famous ML optimizer from Scratch https://www.vizuaranewsletter.com/p/adam-the-most-famous-ml-optimizer

193. ADAM Optimizer Explained Step by Step - First & Second ... https://www.youtube.com/watch?v=5tygLBoN_io

194. How do I determine why my optimizer is taking up so much ... https://discuss.pytorch.org/t/how-do-i-determine-why-my-optimizer-is-taking-up-so-much-memory/199900

195. 8-bit adam vs adafactor vs prodigy, which optimizer is better? https://www.reddit.com/r/StableDiffusion/comments/17mk1hs/8bit_adam_vs_adafactor_vs_prodigy_which_optimizer/

196. PyTorch LLM Memory Optimization https://sebastianraschka.com/blog/2023/pytorch-memory-optimization.html

197. Optimizing Memory Usage in PyTorch Models https://machinelearningmastery.com/optimizing-memory-usage-pytorch-models/

198. A comprehensive guide to memory usage in PyTorch https://medium.com/deep-learning-for-protein-design/a-comprehensive-guide-to-memory-usage-in-pytorch-b9b7c78031d3

199. memory efficient mixed precision optimizers https://arxiv.org/pdf/2309.12381

200. PyTorch vs Burn/Rust - Page 2 - Development https://forum.image.sc/t/pytorch-vs-burn-rust/119776?page=2

201. Torch's `LayerNorm` and Adam optimizer vs those in ... https://github.com/pytorch/pytorch/issues/104857

202. AMD Ryzen, NUMA and numactl on Linux : r/linux_gaming https://www.reddit.com/r/linux_gaming/comments/5yswtr/amd_ryzen_numa_and_numactl_on_linux/

203. Linux Beyond the Basics: NUMA https://medium.com/@weidagang/linux-beyond-the-basics-numa-53b32ff60d6f

204. domargan/awesome-numa https://github.com/domargan/awesome-numa

205. perf: Allocator has a high impact on your Rust programs https://www.reddit.com/r/rust/comments/1riwbqv/perf_allocator_has_a_high_impact_on_your_rust/

206. Rust allocator jemalloc vs mimalloc vs tcmalloc [2026] https://www.kunalganglani.com/blog/rust-allocator-jemalloc-mimalloc-tcmalloc

207. Benchmark with different allocators · Issue #1441 · rust- ... https://github.com/rust-lang/rust-analyzer/issues/1441

208. Optimizing Rust Performance with jemalloc https://dev.to/leapcell/optimizing-rust-performance-with-jemalloc-36lo

209. On Maximizing Your Rust Code's Performance https://jbecker.dev/research/on-writing-performant-rust

210. NUMA, numactl, and taskset: A Practical Guide for AI/ML Engineers https://tonyseah.medium.com/numa-numactl-and-taskset-a-practical-guide-for-ai-ml-engineers-0fbacb450f22

211. Low-Level and NUMA-Aware Optimization for High-Performance ... https://arxiv.org/html/2506.09198v2

212. The Crucial Role of NUMA Awareness in High-Performance Deep Learning https://towardsdatascience.com/the-crucial-role-of-numa-awareness-in-high-performance-deep-learning/

213. Evaluating Uniform Memory Access Mode on AMD's Turin ft. Verda ... https://chipsandcheese.com/p/evaluating-uniform-memory-access

214. How to Configure NUMA Balancing and CPU Pinning on RHEL https://oneuptime.com/blog/post/2026-03-04-numa-balancing-cpu-pinning-rhel-9/view

215. Chapter 4 - Deep Learning via Rust - RantAI https://dlvr.rantai.dev/docs/part-i/chapter-4/

216. Boosting Machine Learning Performance With Rust https://betterprogramming.pub/boosting-machine-learning-performance-with-rust-aab1f3ae1424

217. Optimizer in tch::nn - Rust https://docs.rs/tch/latest/tch/nn/struct.Optimizer.html

218. Deep Learning in Rust: Burn 0.4.0 released and plans for ... https://www.reddit.com/r/rust/comments/101fpkl/deep_learning_in_rust_burn_040_released_and_plans/

219. burn-nn - crates.io: Rust Package Registry https://crates.io/crates/burn-nn

220. Neural Networks https://www.arewelearningyet.com/neural-networks/

221. Burn https://burn.dev/

222. How does cgroups v2 impact Java, .NET, and Node.js in ... https://developers.redhat.com/articles/2025/11/27/how-does-cgroups-v2-impact-java-net-and-nodejs-openshift-4

223. Respect cgroup v2 memory limits on Linux · Issue #234 https://github.com/tukaani-project/xz/issues/234

224. How to Use cgroups v2 to Limit CPU and Memory for ... https://oneuptime.com/blog/post/2026-03-04-use-cgroups-v2-to-limit-cpu-and-memory-for-individual-processes/view

225. Understanding Rust and Kubernetes Resource Management https://openobserve.ai/blog/understanding-kubernetes-resource-management-using-rust-in-containers/

226. AMD Zen 5 Core Architecture Breakdown At Hot Chips https://www.reddit.com/r/Amd/comments/1f2lzb6/amd_zen_5_core_architecture_breakdown_at_hot/

227. Leaked AMD Zen 6 benchmarks confirm huge performance ... https://overclock3d.net/news/cpu_mainboard/leaked-amd-zen-6-benchmarks-confirm-huge-performance-gains-over-zen-5/

228. Unveiling AMD's Zen 5 9000 Series CPUs https://vrlatech.com/unveiling-amds-zen-5-9000-series-cpus/?srsltid

229. AMD Zen 5 Architecture and XDNA 2 Announced https://www.storagereview.com/news/amd-zen-5-architecture-and-xdna-2-announced
230. AMD Reveals More Zen 5 CPU Core Details Review https://www.phoronix.com/review/amd-zen-5-core
231. AMD "Zen" Core Architecture https://www.amd.com/en/technologies/zen-core.html
232. CPU-based algorithm trains deep neural nets up to 15 ... https://news.ycombinator.com/item?id=26745326
233. How to run LLMs with less GPU and CPU memory? https://medium.com/data-science-in-your-pocket/how-to-run-llms-in-less-gpu-and-cpu-memory-6989e6ec5621
234. Adam — PyTorch 2.13 documentation https://docs.pytorch.org/docs/stable/generated/torch.optim.Adam.html
235. A Comparative Study of Optimization Techniques on the ... https://www.scirp.org/pdf/ojop2024133_12730372.pdf
236. A Comparative Study of Optimization Techniques on the ... https://www.researchgate.net/publication/385696033_A_Comparative_Study_of_Optimization_Techniques_on_the_Rosenbrock_Function
237. kogolobo/optimizer_comparison https://github.com/kogolobo/optimizer_comparison
238. A Comparative Study of Optimizers' Performance in Deep ... https://arxiv.org/html/2509.08499v1
239. CPU RAM Usage Keeps Growing as Training One Cycle https://forums.fast.ai/t/cpu-ram-usage-keeps-growing-as-training-one-cycle/30879
240. GPU MODE Lecture 6: Optimizing Optimizers in PyTorch https://christianjmills.com/posts/cuda-mode-notes/lecture-006/
241. Optimize PyTorch Performance for Speed and Memory ... https://towardsdatascience.com/optimize-pytorch-performance-for-speed-and-memory-efficiency-2022-84f453916ea6/
242. Why Does Adam Optimizer Need Bias Correction? https://hiya31.medium.com/why-does-adam-optimizer-need-bias-correction-c6cd4aca99f0
243. Learn Adam Optimizer and Bias Correction https://codefinity.com/courses/v2/c72b49e0-c32c-4728-9036-593d70229393/1052ec71-bcce-4a89-a85f-f3d83d8fc9b2/9a027a57-aa2f-4bc1-9604-94eed7f12cff
244. Why is it important to include a bias correction term for the ... https://stats.stackexchange.com/questions/232741/why-is-it-important-to-include-a-bias-correction-term-for-the-adam-optimizer-for
245. I put together a Rust-native, CPU-only implementation of ... https://www.reddit.com/r/LocalLLaMA/comments/1u14kte/i_put_together_a_rustnative_cpuonly/

246. Rust Autograd Engine Zero Dependencies 5.7x Faster ... https://www.linkedin.com/posts/basaanithanaveenkumar_micrograd-backward-rust-activity-7477209260720869377-03lP

247. Practical Sketching Algorithms for Low-Rank Matrix Approximation https://tropp.caltech.edu/papers/TYUC17-Practical-Sketching.pdf

248. simd-optimization | Skills Marketplace https://lobehub.com/skills/rust-works-succinctly-simd-optimization

249. RaptorQ (RFC6330) and performance optimization in Rust https://www.cberner.com/2019/03/30/raptorq-rfc6330-rust-optimization/

250. The Crucial Role of NUMA Awareness in High-Performance ... https://chaimrand.medium.com/the-crucial-role-of-numa-awareness-in-high-performance-deep-learning-99ae3e8eb49a

251. The Effect of NUMA Tunings on CPU Performance https://s3.cern.ch/inspire-prod-files-a/a590116b1ccb18c7b9f09f1350813c37

252. NUMA Memory Performance https://docs.kernel.org/admin-guide/mm/numaperf.html

253. Improving the speed of neural networks on CPUs https://research.google.com/pubs/archive/37631.pdf

254. Unlocking Performance: How NUMA Tuning Can Triple ... https://www.linkedin.com/pulse/numa-technology-best-compute-intensive-application-like-deepak-soni

255. Benchmarks regarding CPU socket settings and enable- ... https://www.reddit.com/r/Proxmox/comments/1ey7z2p/benchmarks_regarding_cpu_socket_settings_and/

256. FINALLY, ZEN 6 Benchmarks Vs Zen 5 Leak!! Zen 5 Refresh ... https://www.youtube.com/watch?v=Jb-rBJsPGnw

257. AMD Zen 5 Is Mind Blowing 40% Faster than Current ... https://www.reddit.com/r/AMD_Stock/comments/1brphxp/amd_zen_5_is_mind_blowing_40_faster_than_current/

258. IMPRESSIVE Zen 5 Benchmarks LEAK | AMD WINS The CPU ... https://www.youtube.com/watch?v=nOih9Kt29F8

259. Alleged AMD Zen 5 Benchmarked In Dual EPYC ES CPU ... https://wccftech.com/amd-zen-5-benchmark-dual-epyc-es-cpu-64-cores-per-chip-up-to-3-85-ghz-faster-than-96-core-genoa/

260. AMD Explained Zen 5 Ryzen 9000 Benchmark ... https://hothardware.com/news/amd-zen-5-benchmark-discrepancies

261. The 6 Most Exciting New Bits From AMD's Big 'Zen 5 ... https://me.pcmag.com/en/processors/24928/the-6-most-exciting-new-bits-from-amds-big-zen-5-processor-push

262. AMD Zen 5 Overview With Ryzen 9000 Series & Ryzen AI ... https://www.phoronix.com/review/amd-zen5-ryzen-9000

263. AMD Ryzen Zen 5 CPUs Unveiled: What Performance ... https://www.reliablesite.net/hosting-news/amd-ryzen-zen-5-cpu-performance/

264. Zen 5 Efficiency Gain in Perspective (HW Unboxed) https://www.reddit.com/r/hardware/comments/1emtd8w/zen_5_efficiency_gain_in_perspective_hw_unboxed/

265. AMD Zen 5 Technical Deep Dive - Machine Learning / AI https://www.techpowerup.com/review/amd-zen-5-technical-deep-dive/7.html

266. CPU performance optimization guide - part 4 https://gpuopen.com/learn/cpu-performance-guide/cpu-performance-guide-part4/

267. Quick Tip: Reducing High CPU Utilization on NSX due to ... https://williamlam.com/2026/07/quick-tip-reducing-high-cpu-utilization-on-nsx-due-to-slow-entropy-from-amd-zen4-5-cpus.html

268. Building a Scalable AI Fine-Tuning Cluster with AMD Ryzen AI ... https://gist.github.com/jwa91/b0fa41d90ceb27f905aa4fd1fdd3dd68

269. Smarter memory control for SQL Server on RHEL with ... https://developers.redhat.com/articles/2025/08/19/smarter-memory-control-sql-server-rhel-cgroup-v2

270. Rosenbrock Function: Benchmark for Optimization https://www.emergentmind.com/topics/rosenbrock-function

271. Rosenbrock Banana Test Function Optimization Calculator https://metricgate.com/docs/rosenbrock-banana-test-function/

272. The Rosenbrock benchmark suite for machine learning https://www.neuraldesigner.com/blog/the-rosenbrock-benchmark-for-machine-learning/

273. Exploring the Performance of Whale Optimization ... http://jurnalapik.id/index.php/jisit/article/download/35/34

274. Tooling for Reproducible Research https://www.ajs.or.at/index.php/ajs/article/view/2052/874

275. PyTorch Memory Tuning | paulbridger.com https://paulbridger.com/posts/pytorch-memory-tuning/

276. PyTorch Memory Management Strategies https://apxml.com/courses/advanced-pytorch/chapter-1-pytorch-internals-autograd/memory-management

277. Faster PyTorch Training by Reducing Peak Memory ... https://lightning.ai/pages/community/tutorial/faster-pytorch-training-by-reducing-peak-memory/

278. Adam — latest trends in deep learning optimization. https://medium.com/data-science/adam-latest-trends-in-deep-learning-optimization-6be9a291375c

279. A Comparative Study of Optimization Techniques in Deep ... https://indjst.org/download-article.php?Article_Unique_Id

280. The Math behind Adam Optimizer https://towardsdatascience.com/the-math-behind-adam-optimizer-c41407efe59b/

281. Adam Optimizer from scratch | Gradient descent made better ... https://www.youtube.com/watch?v=XHF2jW_U3iM

282. Adam https://optimization.cbe.cornell.edu/index.php?title=Adam

283. Oxidizr — Rust application // Lib.rs https://lib.rs/crates/oxidizr

284. mimalloc: A new, high-performance, scalable memory ... https://www.microsoft.com/en-us/research/blog/mimalloc-a-high-performance-scalable-memory-allocator-for-the-modern-era/

285. Zen 5 Benchmarks : r/pcmasterrace https://www.reddit.com/r/pcmasterrace/comments/1emg5wt/zen_5_benchmarks/

286. Zen 5 tested: Mobile core differs considerably from desktop ... https://www.hwcooling.net/en/zen-5-tested-mobile-core-differs-considerably-from-desktop-one/

287. How to Tune Memory and NUMA Performance on RHEL https://oneuptime.com/blog/post/2026-03-04-tune-memory-numa-performance-rhel-9/view

288. AMD offers new adaptive algorithm, Process ... https://wccftech.com/amd-adaptive-algorithm-pan-process-adaptive-autonuma-increase-performance-linux/

289. Benchmarking GPUs for Mixed Precision Training with ... https://blog.paperspace.com/mixed-precision-training-benchmark/

290. [R] CPU algorithm trains deep neural nets up to 15 times ... https://www.reddit.com/r/MachineLearning/comments/mn8r7f/r_cpu_algorithm_trains_deep_neural_nets_up_to_15/

291. Benchmarking and Analyzing Deep Neural Network Training http://www.cs.toronto.edu/ecosystem/papers/TBD-IISWC_18.pdf

292. Train Neural Networks without Draining your Pocket: Precision ... https://mohanarc.substack.com/p/train-neural-networks-without-draining-14b

293. AMD Delivers Breakthrough MLPerf Training 6.0 Results https://www.amd.com/en/blogs/2026/amd-delivers-breakthrough-mlperf-training-6-0-results.html

294. Deep Learning Inference Frameworks Benchmark https://arxiv.org/pdf/2210.04323

295. Deep Learning GPU Benchmarks 2024 - aime.info https://www.aime.info/blog/en/deep-learning-gpu-benchmarks-2024/

296. Neural Network Inference Optimization/Acceleration - Medium https://whatdhack.medium.com/neural-network-inference-optimization-8651b95e44ee

297. Strange performance issues tch-rs in the mnist convolution ... https://www.reddit.com/r/rust/comments/z2rsuk/strange_performance_issues_tchrs_in_the_mnist/

298. Shape Safe Deep Learning in Rust on Apple Silicon - CS231n https://cs231n.stanford.edu/papers/text_file_840593634-egpaper_cvpr17_final%202.pdf

299. https://github.com/NVIDIA/skills/tree/main/skills/... https://github.com/NVIDIA/skills/tree/main/skills/TensorRT-LLM/perf-optimization

300. Improved Gaussian process modeling: Application to ... https://theses.hal.science/tel-03925818/file/2022UPASG063_PETIT_archivage.pdf

301. Memory Latency - AMD Ryzen Presentation Slide vs Real ... https://www.reddit.com/r/Amd/comments/ck8lw2/memory_latency_amd_ryzen_presentation_slide_vs/

302. MSI Releases Memory "Latency Killer" for AMD Ryzen ... https://www.techpowerup.com/329999/msi-releases-memory-latency-killer-for-amd-ryzen-9000-series-cpus-up-to-12-ns-lower-memory-latency

303. This new patch improves Ryzen 9000 latency by over 50% https://www.digitaltrends.com/computing/amd-ryzen-9000-new-bios-fixes-latency/

304. AMD drops Ryzen 9000 inter-core latency by almost 60% ... https://overclock3d.net/news/cpu_mainboard/amd-drops-ryzen-9000-inter-core-latency-by-almost-60-with-agesa-1202-update/

305. Unlocking Peak AI Performance with MLPerf Client v1.0 on ... https://www.amd.com/en/developer/resources/technical-articles/2025/unlocking-peak-ai-performance-with-mlperf-client-on-ryzen-ai-.html

306. AMD Finally Finds Fixes For Improving Inter-Core Latency ... https://wccftech.com/amd-finally-finds-fixes-for-improving-inter-core-latency-on-strix-point-apus/

307. AMD Delays Ryzen 9000: “Did Not Meet Quality ... http://gamersnexus.net/cpus-news/amd-delays-ryzen-9000-did-not-meet-quality-expectations

308. Zen 5 CPUs get better: AMD reportedly quashes severe ... https://www.notebookcheck.net/Zen-5-CPUs-get-better-AMD-reportedly-quashes-severe-latency-plaguing-Ryzen-9000-CPUs.890379.0.html

309. AMD delays its Ryzen 9000 launch due to unspecified ... https://www.tomshardware.com/pc-components/cpus/amd-delays-its-ryzen-9000-launch-due-to-unspecified-quality-issue-new-launch-in-august-chipmaker-pulls-back-all-units-shipped-globally-for-quality-checks

310. Neural Network and AI Bottlenecks - Subrata Goswami - Medium https://whatdhack.medium.com/neural-network-and-ai-bottlenecks-f9c7357b73fa

311. Maximizing Neural Networks Inference Speed on RockChip ... https://www.youtube.com/watch?v=mDRfXNuIMBE

312. Trends in machine learning hardware https://epoch.ai/publications/trends-in-machine-learning-hardware
313. Benchmarking Energy and Latency in TinyML https://arxiv.org/html/2505.15622v1
314. YouTube https://www.youtube.com/watch?v=ELdiQriZCT8
315. Release Notes — Ryzen AI Software 1.8.0 documentation https://ryzenai.docs.amd.com/projects/WinML/en/stable/relnotes.html
316. AI performance research papers https://www.redhat.com/en/artificial-intelligence/research
317. Benchmarking energy consumption and latency for ... https://repository.ubn.ru.nl/bitstream/handle/2066/290044/1/290044.pdf
318. Accelerating ML Workloads using GPU Tensor Cores https://par.nsf.gov/servlets/purl/10539988
319. How to improve rust performance to 1.4 seconds? https://users.rust-lang.org/t/how-to-improve-rust-performance-to-1-4-seconds/95508
320. Rosenbrock function. Shown is SP1 (the expected running ... https://www.researchgate.net/figure/Rosenbrock-function-Shown-is-SP1-the-expected-running-time-or-number-of-function_fig3_221131728
321. Finding the minimum of Rosenbrock's function numerically https://math.stackexchange.com/questions/4661231/finding-the-minimum-of-rosenbrocks-function-numerically
322. Inside AMD's AI Playbook: Strategy, Scale and Execution - Six ... https://www.youtube.com/watch?v=yyfGOXnUEn4
323. Lookahead Optimizer: k steps forward, 1 step back https://arxiv.org/pdf/1907.08610
324. Adam vs. AdamW: Understanding Weight Decay and Its ... https://yassin01.medium.com/adam-vs-adamw-understanding-weight-decay-and-its-impact-on-model-performance-b7414f0af8a1
325. Deep Learning Models on CPUs: A Methodology ... https://www.journal.riverpublishers.com/index.php/JMLTAP/article/view/268
326. AMD Zen 5 Details Emerge with GCC "Znver5" Patch https://www.techpowerup.com/318991/amd-zen-5-details-emerge-with-gcc-znver5-patch-new-avx-instructions-larger-pipelines
327. AMD Responds To Slow Ryzen 9000 Performance | Deep ... https://www.youtube.com/watch?v=DI9clRKVtZw
328. A Closer Look at AMD's Ryzen 9000 Series | (Launch Date ... https://www.youtube.com/watch?v=lpEHvmDPaNc
329. AMD Ryzen 9000 Series Announced - Zen 5 Showing Big ... https://www.phoronix.com/news/AMD-Ryzen-9000-Series-Zen-5

330. DDR5-6400 Confirmed as Sweetspot Speed of Ryzen ... https://www.techpowerup.com/324023/ddr5-6400-confirmed-as-sweetspot-speed-of-ryzen-9000-zen-5-desktop-processors

331. AMD boosts Zen 5 official warrantied TDP to 105W, fixes ... https://www.tomshardware.com/pc-components/cpus/amd-boosts-zen-5-official-warrantied-tdp-to-105w-fixes-ryzen-9000-latency-issue-launches-800-series-chipsets

332. AMD Ryzen 9000 Deep Dive - What does Zen 5 deliver? https://overclock3d.net/reviews/cpu_mainboard/amd-ryzen-9000-deep-dive-what-does-zen-5-deliver/

333. AMD Ryzen 9000 inter-core latency significantly reduced ... https://www.reddit.com/r/Amd/comments/1fit3oi/amd_ryzen_9000_intercore_latency_significantly/

334. New Ryzen 9000 Benchmarks.* Too good to be true? https://www.youtube.com/watch?v=aWWpoVZBhDo

335. AMD vs NVIDIA Inference Benchmark: Who Wins? https://newsletter.semianalysis.com/p/amd-vs-nvidia-inference-benchmark-who-wins-performance-cost-per-million-tokens

336. https://scholar.google.com/citations?view_op=view_... https://scholar.google.com/citations?view_op=view_citation&hl=en&user=sEMrGicAAAAJ&citation_for_view=sEMrGicAAAAJ:uc_IGeMz5qoC

337. CPU selection and compilation speed - help https://users.rust-lang.org/t/cpu-selection-and-compilation-speed/121006

338. Exploring SIMD in Rust https://hugotunius.se/2017/12/01/exploring-simd-in-rust.html

339. Rust developers might consider switching to jemallocator ... https://news.ycombinator.com/item?id=38459571

340. Surrogate-Based Optimization Techniques for Process ... https://arxiv.org/html/2412.13948v1

341. Accelerated optimization in deep learning with a ... https://www.nature.com/articles/s41467-024-54451-3

342. optimization via strategic law of large numbers https://cowles.yale.edu/sites/default/files/2025-11/d2471.pdf

343. Why Thermal Throttling Is Slowing Down Your GPUs https://techround.co.uk/artificial-intelligence/hidden-bottleneck-ai-boom-thermal-throttling-slowing-down-gpus/

344. Unlocking the AMD Neural Processing Unit for ML Training ... https://arxiv.org/pdf/2504.03083

345. Nvidia Blackwell Reigns Supreme in MLPerf Training ... https://spectrum.ieee.org/nvidia-blackwell-mlperf-training-5

346. AMD GPUs go brrr https://hazyresearch.stanford.edu/blog/2025-11-09-amd-brr

347. The Sleeping Giant Wakes: Why AMD's MLPerf ... https://techspective.net/2026/04/06/the-sleeping-giant-wakes-why-amds-mlperf-breakthrough-signals-the-beginning-of-the-end-for-nvidias-ai-monopoly/

348. Enhancing Adam Optimizer with Gradient Entropy https://pub.aimind.so/enhancing-adam-with-gradient-entropy-for-optimizer-regularization-c1f05248c980

349. Tch-rs - Rust bindings for the C++ api of PyTorch. https://github.com/laurentmazare/tch-rs

350. 24 AMD CPUs Benchmark in Rust | Benchmark 24 CPUs ... https://www.youtube.com/watch?v=0KfaMmTbtCw

351. Benchmarking Neural Network Training Algorithms https://arxiv.org/abs/2306.07179

352. Default musl allocator considered harmful (to performance) https://nickb.dev/blog/default-musl-allocator-considered-harmful-to-performance/

353. Unveiling AMD's Zen 5 9000 Series CPUs https://vrlatech.com/unveiling-amds-zen-5-9000-series-cpus/?srsltid=AfmBOopoK-wQzQ8Y7lP3zbaTUCqPcoJvN5dxL8Xf2p2AuP6yOz7OVnwC

354. AMD Unveils Next-Gen “Zen 5” Ryzen Processors to Power ... https://www.amd.com/en/newsroom/press-releases/2024-6-2-amd-unveils-next-gen-zen-5-ryzen-processors-to-p.html

355. AMD Zen 5 Technical Deep Dive - Architecture https://www.techpowerup.com/review/amd-zen-5-technical-deep-dive/3.html

356. AMD Zen 5 CPU Core Architecture Allegedly More Than 40 ... https://wccftech.com/amd-zen-5-cpu-core-architecture-over-40-percent-faster-than-zen-4/

357. 'Zen 5' Microarchitecture Explained: Here Comes the Fast, ... https://www.pcmag.com/news/zen-5-microarchitecture-explained-here-comes-the-fast-efficient-amd-ryzen

358. Zen 4 vs Zen 5 - What's the deal? https://pcpartpicker.com/forums/topic/468360-zen-4-vs-zen-5-whats-the-deal

359. Adam Optimizer: The Game Changer in Deep Learning https://www.youtube.com/watch?v=8b5yseiyg28

360. Rank-one convexification for quadratic ... https://optimization-online.org/wp-content/uploads/2025/04/main.pdf

361. Improved Gaussian process modeling. Application to ... https://www.theses.fr/2022UPASG063.pdf

362. Track: Poster Session 1 https://neurips.cc/virtual/2023/session/74069

363. Simplifying Adam: Bias Correction Debunked https://arxiv.org/html/2511.20516v2

364. (PDF) Adam Simplified: Bias Correction Simplified https://www.researchgate.net/publication/397983086_Adam_Simplified_Bias_Correction_Simplified
365. Utilization - pytorch-optimizer https://pytorch-optimizers.readthedocs.io/en/latest/util/
366. arXiv:2007.06558v4 [stat.ML] 24 Sep 2020 https://web.stanford.edu/~chen96/papers/EntropyNPG.pdf
367. Puget Systems Podcast - AMD Ryzen 9000 Series Benchmark ... https://www.youtube.com/watch?v=NN69YWv6AtM
368. Ryzen 9000 memory improvements? : r/Amd https://www.reddit.com/r/Amd/comments/1evzi2b/ryzen_9000_memory_improvements/
369. Autodiff benchmark comparison · Issue #1559 · tracel-ai/burn https://github.com/tracel-ai/burn/issues/1559
370. AMD Ryzen 9000 vs. Intel Core Ultra Arrow Lake On Linux ... https://www.phoronix.com/review/ryzen9000-core-ultra-linux613/12
371. Ryzen 9000 Review: Productivity Focus Benchmarks https://www.youtube.com/watch?v=KYE7Q1ro3Xw
372. Ryzen 9000 Performancy/Efficiency Discussion : r/Amd https://www.reddit.com/r/Amd/comments/1g15fkn/ryzen_9000_performancyefficiency_discussion/
373. AMD Ryzen 9000 Content Creation Review https://www.pugetsystems.com/labs/articles/amd-ryzen-9000-content-creation-review/
374. AMD Ryzen 9000 vs. Intel Core Ultra Arrow Lake On Linux ... https://www.phoronix.com/review/ryzen9000-core-ultra-linux613/15
375. AMD Ryzen 9000 review: Impressive efficiency, with bugs ... https://arstechnica.com/gadgets/2024/08/amd-ryzen-9000-review-impressive-efficiency-with-bugs-and-so-so-speed-boosts/
376. Adam with Corrected Weight Decay (AdamC) https://www.emergentmind.com/topics/adam-with-corrected-weight-decay-adamc
377. ON THE PITFALLS OF WEIGHT DECAY DECOUPLING https://hmdliu.site/assets/pdf/Report_AMLDS.pdf
378. AdamW: Decoupled Weight Decay for Neural Networks https://mbrenndoerfer.com/writing/adamw-optimizer-decoupled-weight-decay
379. Adam Simplified: Bias Correction Debunked https://arxiv.org/pdf/2511.20516
380. Adam Optimizer https://medium.com/@AILearningHub/adam-optimizer-8f35f038601b
381. an uncertainty-based objective function for https://elib.dlr.de/197663/1/Hybrid_Loss_PL.pdf

382. Numerical Optimization https://www.ime.unicamp.br/~pulino/MT404/TextosOnline/NocedalJ.pdf

383. Optimized step size control within the Rosenbrock solvers ... https://gmd.copernicus.org/articles/18/4273/2025/gmd-18-4273-2025.pdf

384. cgroup-memory - crates.io: Rust Package Registry https://crates.io/crates/cgroup-memory

385. Rust and RAII Memory Management - Computerphile : r/rust https://www.reddit.com/r/rust/comments/119xib6/rust_and_raii_memory_management_computerphile/

386. Building a minimal memory allocator in Rust and testing it with ... https://www.youtube.com/watch?v=ESzmAgaT3CU

387. Universal guide to configuring all Ryzen 9000 CPUs ... https://www.reddit.com/r/ryzen/comments/1gq1yu9/universal_guide_to_configuring_all_ryzen_9000/

388. Unlocking the Secrets of AMD Zen 4 P Cores: Maximum ... https://www.youtube.com/watch?v=3_PgyZjkow4

389. Preface to the CPU performance optimization guide https://gpuopen.com/learn/cpu-performance-guide/cpu-performance-guide-preface/

390. AMD https://huggingface.co/amd/collections

391. GaLore: Memory-Efficient LLM Training by Gradient Low ... https://arxiv.org/abs/2403.03507

392. GaLore: Memory-Efficient LLM Training by Gradient Low-Rank ... https://www.youtube.com/watch?v=2_6aHjHIcC4